

**OikoNaos**  
**Test Case Specification**  
**Versione 1.5**



Data: 21/12/2025

|                                    |                  |
|------------------------------------|------------------|
| Progetto: OikoNaos                 | Versione: 1.5    |
| Documento: Test Case Specification | Data: 21/12/2025 |

**Coordinatore del progetto:**

| Nome          | Matricola  |
|---------------|------------|
| Luigi Potestà | 0512120076 |

**Partecipanti:**

| Nome                      | Matricola  |
|---------------------------|------------|
| Luigi Potestà             | 0512120076 |
| Maria Antonietta Ruggiero | 0512121126 |
| Giulia Buonafine          | 0512119695 |
| Mario Sinopoli            | 0512119113 |

|                    |                           |
|--------------------|---------------------------|
| <b>Scritto da:</b> | Maria Antonietta Ruggiero |
|--------------------|---------------------------|

**Revision History**

| Data       | Versione | Descrizione                                                                                                                                         | Autore                                      |
|------------|----------|-----------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------|
| 28/11/2025 | 1.0      | Creazione della struttura del documento (Indice), stesura delle sezioni introduttive (Purpose, Audience) e Test Case Identifier.                    | Luigi Potestà                               |
| 02/12/2025 | 1.1      | Analisi dei requisiti per la definizione dei Test Items e stesura completa delle specifiche di Input e Output (Casi positivi, negativi e boundary). | Giulia Buonafine                            |
| 04/12/2025 | 1.2      | Definizione dell'ambiente di test (Environmental Needs), requisiti procedurali (Setup/Teardown) e script di inizializzazione DB.                    | Mario Sinopoli                              |
| 06/12/2025 | 1.3      | Stesura delle dipendenze tra casi di test (Intercase Dependencies), revisione finale di coerenza con RAD/SDD e rilascio versione definitiva.        | Maria Antonietta Ruggiero                   |
| 11/12/2025 | 1.4      | Ridefinizione Input e Output Specification in relazione al codice implementato                                                                      | Mario Sinopoli<br>Maria Antonietta Ruggiero |
| 21/12/2025 | 1.5      | Migliorie nelle sezioni Special procedural requirements e Intercase dependencies                                                                    | Giulia Buonafine<br>Luigi Potestà           |

## Indice

|                                            |    |
|--------------------------------------------|----|
| 1. Purpose .....                           | 4  |
| 2. Audience .....                          | 4  |
| 3. Test case specification identifier..... | 5  |
| 4. Test items .....                        | 5  |
| 5. Input specification .....               | 6  |
| 6. Output specification.....               | 7  |
| 7. Environmental needs .....               | 8  |
| 8. Special procedural requirements .....   | 9  |
| 9. Intercase dependencies .....            | 10 |

# 1. Purpose

Lo scopo del presente documento è definire in modo dettagliato le specifiche dei casi di test (*Test Cases*) per il sistema OikoNaos, al fine di verificare la corretta implementazione delle funzionalità previste per la versione prototipale del sistema, così come descritte nel Requirements Analysis Document (RAD) e progettate nel System Design Document (SDD) e nell'Object Design Document (ODD).

Il documento intende fornire una guida strutturata per l'esecuzione delle attività di testing, assicurando che i principali flussi operativi implementati nel prototipo funzionino correttamente. In particolare, i casi di test sono finalizzati a verificare il corretto funzionamento delle funzionalità di autenticazione degli utenti, la gestione delle sessioni applicative e il meccanismo di controllo degli accessi alle risorse protette, implementato tramite filtro di autenticazione, garantendo che solo gli utenti autorizzati possano accedere alle funzionalità riservate.

Le attività di verifica riguardano inoltre la gestione delle prenotazioni degli spazi comuni, includendo la creazione, la consultazione e la cancellazione delle prenotazioni, nonché il corretto funzionamento dei meccanismi di persistenza dei dati e il rispetto dei vincoli di integrità definiti a livello progettuale. Analogamente, i casi di test coprono la creazione e la consultazione dei ticket di manutenzione da parte dei coinquilini e le funzionalità di gestione dei ticket riservate al ruolo di Supervisore, limitatamente alla presa in carico delle segnalazioni e all'aggiornamento dello stato del ticket.

Infine, il documento mira a verificare il rispetto delle principali regole di business e dei meccanismi di sicurezza previsti dal sistema, in particolare il controllo degli accessi basato sui ruoli (RBAC), assicurando la protezione delle informazioni sensibili e la coerenza complessiva tra requisiti, architettura e implementazione. L'esecuzione dei casi di test descritti consentirà di valutare la qualità del prototipo prima del rilascio, individuando eventuali difetti o discrepanze rispetto alle specifiche progettuali.

## 2. Audience

Il presente documento è destinato alle diverse figure coinvolte nel ciclo di vita del progetto OikoNaos e rappresenta un riferimento condiviso per le attività di verifica e validazione del sistema. In particolare, esso è rivolto al **Team di Testing**, responsabile dell'esecuzione dei casi di test descritti, della raccolta e registrazione dei risultati ottenuti e della segnalazione delle anomalie riscontrate durante le fasi di verifica. Il documento è inoltre destinato al **Team di Sviluppo**, che può utilizzarlo per comprendere in modo chiaro gli scenari di test che il codice implementato è chiamato a soddisfare e per analizzare i difetti individuati, al fine di procedere alla loro correzione e alla successiva verifica di regressione.

Un ulteriore destinatario del documento è il **Project Manager**, il quale può fare riferimento alle specifiche dei casi di test per monitorare l'avanzamento delle attività di testing, valutare la copertura dei requisiti rispetto alle funzionalità implementate e stimare il livello complessivo di qualità del rilascio software.

Infine, il documento si rivolge anche all'**Azienda Coordinatrice**, in qualità di stakeholder del progetto, fornendo una base di riferimento per i test di accettazione utente, al fine di verificare che il prototipo soddisfi le esigenze operative e i requisiti di business previsti.

### 3. Test case specification identifier

Il presente documento è identificato dal codice **TCS-OikoNaos-1.5**. Tale identificativo è utilizzato per garantire la tracciabilità del documento all'interno della documentazione di progetto e per distinguerlo dalle altre specifiche prodotte nelle diverse fasi del ciclo di sviluppo del sistema.

### 4. Test items

Gli elementi oggetto di test corrispondono ai moduli software effettivamente implementati nella versione prototipale del sistema OikoNaos, come definiti nel System Design Document e dettagliati nell'Object Design Document. Tale paragrafo elenca esclusivamente i sottosistemi inclusi nel prototipo e oggetto delle attività di verifica descritte nel Test Plan, escludendo i moduli previsti per evoluzioni future ma non implementati nella versione corrente.

#### **Modulo Gestione Autenticazione**

Il modulo di Gestione Autenticazione fa riferimento al sottosistema omonimo descritto nel System Design Document ed è dettagliato nel package di autenticazione dell'Object Design Document. Le funzionalità verificate attraverso i casi di test corrispondono ai requisiti funzionali RF01–RF04 definiti nel Requirements Analysis Document.

Le attività di test riguardano il corretto funzionamento delle operazioni di login e logout degli utenti, la gestione delle sessioni applicative e il controllo degli accessi alle risorse protette del sistema. In particolare, viene verificato che le funzionalità riservate siano accessibili esclusivamente a utenti autenticati, in conformità con il meccanismo di protezione delle risorse previsto dall'architettura del sistema.

#### **Modulo Gestione Prenotazioni degli Spazi Comuni**

Il modulo di Gestione Prenotazioni fa riferimento al sottosistema di Gestione Prenotazioni descritto nel System Design Document ed è dettagliato nel relativo package dell'Object Design Document. Le funzionalità verificate corrispondono ai requisiti funzionali RF08 e RF09 definiti nel Requirements Analysis Document.

I casi di test relativi a questo modulo verificano la possibilità per gli utenti autenticati di consultare le prenotazioni esistenti, creare nuove prenotazioni per gli spazi comuni e cancellare prenotazioni già effettuate. Le attività di verifica includono inoltre il controllo della disponibilità degli spazi, la prevenzione di conflitti temporali e la coerenza dei dati memorizzati nel database.

#### **Modulo Gestione Ticket di Manutenzione**

Il modulo di Gestione Ticket di Manutenzione fa riferimento al sottosistema di Gestione Ticket descritto nel System Design Document ed è dettagliato nel package Ticket dell'Object Design Document. Le funzionalità oggetto di test sono riconducibili ai requisiti funzionali RF10–RF12 definiti nel Requirements Analysis Document.

Le attività di test includono la creazione e la consultazione dei ticket di manutenzione da parte dei coinquilini, verificando la corretta registrazione delle segnalazioni e la visualizzazione dello stato associato. Sono inoltre oggetto di verifica le funzionalità riservate al ruolo di Supervisore, limitatamente alla presa in carico delle segnalazioni e all'aggiornamento dello stato del ticket,

assicurando che tali operazioni siano consentite esclusivamente a utenti autorizzati e che le modifiche vengano correttamente persistite.

### **Moduli esclusi dal perimetro di test**

I sottosistemi relativi alla Gestione Eventi e Bacheca, alla Gestione delle Risorse Condivise, alla Gestione delle Spese e dei Pagamenti e alla gestione avanzata del Profilo Utente, pur essendo previsti nel Requirements Analysis Document e descritti a livello architetturale, non sono inclusi nel perimetro di test della versione prototipale in quanto non implementati nel codice corrente. Le funzionalità associate a tali moduli saranno oggetto di specifiche attività di verifica nelle successive evoluzioni del sistema.

## **5. Input specification**

Questa sezione definisce le tipologie di dati di ingresso necessari per l'esecuzione dei casi di test previsti per il sistema OikoNaos. Le specifiche di input sono finalizzate a descrivere i valori forniti al sistema dagli utenti durante l'interazione con le funzionalità implementate nel prototipo, al fine di verificare sia il comportamento corretto del sistema sia la gestione delle condizioni di errore. Le specifiche di input si suddividono in due categorie principali: input validi, costituiti da dati corretti che soddisfano le precondizioni dei casi di test e consentono il completamento delle operazioni previste, e input non validi o eccezionali, costituiti da dati errati, incompleti o non coerenti, utilizzati per verificare la robustezza del sistema e la corretta gestione degli errori. Di seguito sono riportati alcuni esempi rappresentativi per ciascuna categoria.

### **Esempio di input valido (caso: creazione di una prenotazione)**

Riferimento: Scenario "Prenotazione spazio comune" – *Use Case UC07*

Nel contesto considerato, un coinquilino autenticato intende effettuare una prenotazione per uno spazio comune disponibile. L'utente ha già effettuato correttamente l'accesso al sistema ed è associato al ruolo di Coinquilino.

Un esempio di input valido fornito al sistema è il seguente:

- Utente autenticato: luca.rossi@example.com
- Spazio comune selezionato: Sala Studio
- Data prenotazione: 11/10/2025
- Intervallo orario: 12:00 – 15:00

L'obiettivo del test è verificare che il sistema accetti correttamente i dati inseriti, consenta la creazione della prenotazione e registri l'operazione in modo coerente nel database.

### **Esempio di input non valido (caso: login fallito)**

Riferimento: Scenario "Autenticazione" – *Use Case UC03*

In questo scenario un utente tenta di accedere al sistema utilizzando credenziali non corrette. L'utente non risulta autenticato e si trova nella pagina di login.

Un esempio di input non valido è il seguente:

- Username: matteob23
- Password: PasswordErrata123

L'obiettivo del test è verificare che il sistema rifiuti l'accesso, non crei una sessione applicativa e mostri un messaggio di errore appropriato, in conformità con quanto previsto dal requisito funzionale *RF02*.

### Esempio di input di tipo limite (Boundary Testing)

Riferimento: Scenario “Prenotazione spazio comune” – Use Case *UC07*

Il caso di test considera una situazione limite in cui un coinquilino tenta di prenotare uno spazio comune per un intervallo temporale che rappresenta l’ultima disponibilità residua per lo spazio selezionato.

Un esempio di input di tipo limite è il seguente:

- Utente autenticato: luca.rossi@example.com
- Spazio comune selezionato: Sala Studio
- Data prenotazione: 12/12/2025
- Intervallo orario: 15:00 – 18:00

L’obiettivo del test è verificare che il sistema gestisca correttamente la condizione di disponibilità minima, consentendo la prenotazione dell’ultima disponibilità ed evitando richieste successive in conflitto per lo stesso intervallo temporale.

## 6. Output specification

Questa sezione descrive i risultati attesi dal sistema in risposta agli input specificati nella sezione precedente. Un test è considerato *superato (Pass)* esclusivamente quando l’output prodotto dal sistema coincide con l’output atteso, sia in termini di risposta dell’interfaccia utente sia in termini di variazioni dello stato interno del sistema e dei dati persistenti.

Le specifiche di output coprono pertanto il comportamento osservabile dall’utente e gli effetti sulle informazioni memorizzate, garantendo la coerenza tra interazione applicativa, logica di business e persistenza dei dati.

### Esempio di output atteso – Caso positivo (creazione di una prenotazione)

Riferimento: Verifica requisito *RF08* (Use Case *UC07*)

A fronte di un input valido per la creazione di una prenotazione, il sistema deve fornire un riscontro positivo all’utente, confermando l’avvenuta operazione. L’interfaccia utente visualizza un messaggio di conferma dell’operazione e aggiorna la vista delle prenotazioni in modo da riflettere la nuova prenotazione effettuata.

Dal punto di vista dello stato del sistema, viene registrata una nuova prenotazione persistente associata all’utente autenticato, alla risorsa selezionata, alla data e all’intervallo temporale specificati. La logica di business aggiorna la disponibilità dello spazio prenotato, impedendo la creazione di ulteriori prenotazioni in conflitto per lo stesso intervallo temporale.

### Esempio di output atteso – Caso negativo (login fallito)

Riferimento: Verifica requisito *RF02* (Use Case *UC03*)

In caso di inserimento di credenziali non corrette durante il tentativo di autenticazione, il sistema deve rifiutare l’accesso. L’interfaccia utente visualizza un messaggio di errore che informa l’utente dell’esito negativo dell’operazione e mantiene l’utente nella pagina di login, impedendo l’accesso alle funzionalità riservate. Dal punto di vista dello stato del sistema, non viene creata alcuna sessione applicativa e i dati persistenti rimangono invariati. La logica di business verifica la non corrispondenza delle credenziali fornite e applica correttamente le regole di sicurezza previste, negando l’autenticazione dell’utente.

## Esempio di output atteso – Caso limite (Boundary Testing)

Riferimento: Verifica requisito *RF08* (Use Case *UC07*)

Nel caso di prenotazione relativa all'ultima disponibilità residua di uno spazio comune per un determinato intervallo temporale, il sistema deve consentire correttamente l'operazione. L'interfaccia utente fornisce una conferma dell'avvenuta prenotazione e aggiorna le informazioni visualizzate in modo da indicare che lo spazio non è più disponibile per l'intervallo selezionato.

A livello di persistenza, viene registrata la prenotazione associata all'utente e alla risorsa, portando la disponibilità per quello specifico intervallo temporale a esaurimento. La logica di business deve garantire che qualsiasi tentativo successivo di prenotazione per la stessa risorsa, data e intervallo temporale venga rifiutato o segnalato come non disponibile, prevenendo situazioni di sovrapposizione o incoerenza.

## 7. Environmental needs

Questa sezione definisce i requisiti hardware e software dell'ambiente di test (Test Environment) necessario per l'esecuzione dei casi di test. L'ambiente di test deve replicare, per quanto possibile, l'ambiente di produzione descritto nel System Design Document (Sez. 5.3) e rispettare i vincoli di implementazione e i requisiti non funzionali indicati nel Requirements Analysis Document (RNF12 – RNF15). L'obiettivo è garantire che i risultati dei test siano affidabili, ripetibili e rappresentativi del comportamento del sistema in condizioni operative reali.

### Hardware Requirements

**Server:** È richiesta una macchina fisica o virtuale configurata per ospitare l'Application Server e il DBMS, con adeguato supporto alla connettività di rete TCP/IP e risorse sufficienti per eseguire l'applicazione web e il database in modo stabile durante le sessioni di test.

**Client:** Devono essere disponibili dispositivi eterogenei (desktop/laptop e, ove possibile, dispositivi mobili) per verificare la corretta fruizione dell'interfaccia web e l'esperienza utente su differenti configurazioni, in coerenza con i requisiti di compatibilità e accessibilità previsti.

### Software Requirements

**Application Server:** Apache Tomcat 10.1, configurato per l'esecuzione dell'applicazione web basata su tecnologia Java/Jakarta EE.

**Database Server.** MySQL 8.0 o superiore, utilizzato per la persistenza dei dati relativi alle funzionalità implementate (autenticazione, prenotazioni, ticket).

**Ambiente Runtime:** Java Development Kit (JDK) versione 17, coerente con lo stack di progetto e con la compilazione/esecuzione dell'applicazione.

**Browser Web (Client):** Google Chrome, Mozilla Firefox, Microsoft Edge o Apple Safari, preferibilmente aggiornati, al fine di verificare la compatibilità lato client come previsto dai vincoli di implementazione.

**Driver Database:** Driver JDBC compatibile con MySQL 8, necessario per la corretta connessione dell'applicazione al DBMS e per l'esecuzione delle operazioni di persistenza durante i test.

## Tools di Supporto

**IDE:** IntelliJ IDEA per l'esecuzione e il debug delle suite di test automatici (es. JUnit) e per l'analisi dei difetti.

**Version Control:** Git, utilizzato per il recupero e la gestione della versione del codice sorgente oggetto di test, garantendo tracciabilità delle modifiche e riproducibilità delle build considerate "stabili".

## 8. Special procedural requirements

Questa sezione descrive le procedure di configurazione, le precondizioni di sicurezza e le attività di gestione dei dati necessarie per garantire la validità, la ripetibilità e l'affidabilità dei casi di test previsti per il sistema OikoNaos.

### Configurazione dei dati (Data Seeding)

Prima dell'esecuzione dei casi di test, il database deve essere inizializzato con un insieme di dati di test coerente con il perimetro funzionale del prototipo. In particolare, l'inizializzazione deve prevedere la presenza di utenti di test già registrati, associati ai ruoli previsti dal sistema, al fine di consentire la verifica delle funzionalità di autenticazione, prenotazione e gestione dei ticket.

Il dataset iniziale deve includere almeno:

- Uno o più utenti con ruolo di Coinquilino, utilizzabili per i test di autenticazione, prenotazione e creazione dei ticket;
- Uno o più utenti con ruolo di Supervisore, necessari per la verifica delle funzionalità di gestione avanzata dei ticket;
- Eventuali prenotazioni e ticket preesistenti, utili per testare scenari di consultazione, conflitto e aggiornamento.

Le password degli utenti di test devono essere archiviate nel database utilizzando un meccanismo di hashing coerente con le politiche di sicurezza adottate dal sistema, in conformità con i requisiti non funzionali definiti nel RAD.

### Configurazione della sicurezza

L'ambiente di test deve essere configurato in modo da consentire la verifica dei meccanismi di sicurezza implementati nel prototipo. In particolare, il sistema deve supportare la gestione delle sessioni applicative e il controllo degli accessi alle risorse protette, assicurando che le funzionalità riservate siano accessibili esclusivamente agli utenti autenticati e autorizzati. Devono essere disponibili account di test associati ai ruoli previsti dal modello di controllo degli accessi basato sui ruoli (Role-Based Access Control), in particolare i ruoli di Coinquilino e Supervisore, al fine di verificare la corretta applicazione delle restrizioni di accesso e delle funzionalità riservate.

### Procedura di ripristino (Teardown)

Al termine di ciascuna sessione di test, il sistema deve essere riportato a uno stato consistente e noto, al fine di garantire la ripetibilità delle verifiche successive. Le attività di ripristino includono la rimozione o il reset dei dati creati durante l'esecuzione dei test, quali prenotazioni e ticket inseriti a fini di verifica. Le operazioni di pulizia possono essere eseguite tramite script di database o procedure

equivalenti, purché consentano di ripristinare il dataset iniziale utilizzato per i test, evitando interferenze tra sessioni di verifica consecutive.

## 9. Intercase dependencies

Questa sezione definisce le dipendenze logiche e temporali tra i casi di test previsti per il sistema OikoNaos. L'ordine di esecuzione dei test è vincolante, poiché l'output prodotto da un caso di test costituisce la preconditione (Entry Condition) per l'esecuzione dei casi di test successivi. La corretta gestione delle dipendenze garantisce la validità dei risultati e la coerenza dello stato del sistema durante le attività di verifica.

### Dipendenze di autenticazione (bloccanti)

**Pre-requisito:** Prima dell'esecuzione dei test funzionali, deve essere disponibile almeno un utente valido nel sistema. Tale condizione può essere soddisfatta tramite il precaricamento di utenti di test nel database, in conformità con le procedure di inizializzazione dei dati descritte nella sezione relativa al Data Seeding.

**Vincolo:** Il superamento del test di autenticazione (login) rappresenta un prerequisito fondamentale per l'esecuzione di tutti i casi di test relativi alle funzionalità riservate agli utenti autenticati, quali la gestione delle prenotazioni e dei ticket di manutenzione. I casi di test successivi non devono essere eseguiti in assenza di una sessione applicativa valida.

### Dipendenze funzionali (workflow applicativo)

**Gestione Prenotazioni:** I casi di test relativi alla cancellazione o alla consultazione di una prenotazione dipendono dal completamento con successo di un caso di test di creazione di una prenotazione. In assenza di una prenotazione esistente, tali test non risultano eseguibili.

**Gestione Ticket:** I casi di test relativi alle funzionalità di gestione dei ticket riservate al ruolo di Supervisore, quali la presa in carico e l'aggiornamento dello stato del ticket, dipendono dal completamento con successo del caso di test di creazione di un ticket da parte di un coinquilino. I test di gestione avanzata non devono essere eseguiti se non è presente almeno un ticket valido nel sistema.

### Dipendenze legate allo stato del sistema (disponibilità e casi limite)

I casi di test relativi a situazioni di concorrenza e ai casi limite (Boundary Testing) nella gestione delle prenotazioni richiedono che lo stato del sistema e del database rifletta una condizione iniziale nota e controllata prima dell'esecuzione del test. In particolare, il dataset di test deve essere configurato in modo da rappresentare una disponibilità residua minima per uno specifico intervallo temporale, al fine di verificare il corretto comportamento del sistema in condizioni di disponibilità limitata e di garantire il rispetto dei vincoli di integrità definiti a livello di business logic.